算分第2次作业

郭劲豪 2200013146@stu.pku.edu.cn

习题2

2.7

(1)

将A数组内的数从小到大用$\Theta(n\log n)$的算法排序，然后选择中位数，判断中位数出现次数是否大于$\frac{n}{2}$即可。

(2)

将A中元素分成5个一组，求出每一组中的中位数，然后将中位数序列抽取出来选择中位数，再根据这个选择出来的中位数值x将序列划分成2部分，若当前值是整个序列的中位数，那么返回；否则递归地处理剩下的序列。

这样子的时间复杂度是：

$T(n)=T(\frac{n}{5})+T(\frac{7n}{10})+O(n)=T(n)$.

最后判断中位数出现次数是否大于$\frac{n}{2}$即可。

(3)

考虑从第一个到最后一个元素将整个A数组扫描一遍，同时记录当前的主元素和主元素剩余数。我们将主元素和非主元素的其他元素两两配对，那么因为主元素出现次数大于n/2，所以最后一定会有剩余。

那么我们在扫描的过程中，遇到一个和当前主元素不相同的元素，就将主元素剩余数-1；

如果主元素剩余数<0，那么就将这个元素替换为主元素。

如果存在主元素，那么最后剩下的元素就是主元素。只需要将整个A数组扫描一次判断他的出现次数是不是大于$\frac{n}{2}$。这样子的复杂度是$\Theta(n)$的。

2.12

把B数组排序，然后对于A的每个元素在B中进行二分查找。

时间复杂度为：$T(n)=O(\log n\log(\log n))+O(n\log\log n)=O(n\log\log n)$.

伪代码如下：

```
sort(B + 1, B + m + 1);
for i in 1..n:
    if(BinarySearch(A[i], B + 1, B + m + 1)) C[++tot] = A[i];
```

最后C中存储的就是答案。

2.15

(1)

考虑`Findmax`函数是$O(n)$的，所以A算法的时间复杂度为$O(n*i)$，最坏情况下当$i=\sqrt{n}$的时候达到$O(n\sqrt{n})$

对于算法B考虑，算法的复杂度瓶颈在于将S数组排序，所以算法复杂度最坏为$O(n\log n)$的。

(2)

考虑我们选取出S数组中的第i大元素，然后将剩下的比它大的数都放在他的右边，同时排序从大到小输出即可。

复杂度为$T(n)=O(n)+O(\sqrt{n}\log\sqrt{n})=O(n)$

伪代码如下：

```
x = quick_Seleck(S, i); // 选取第i大的数
Swap(S, x, i); // 将比x大的数交换到i前面
sort(S + 1, S + i + 1); // 将前i个数排序
Print(S, i);
```

2.16

(1)

将S中的数扫描一遍，求出最大值和最小值，然后作为x和y即可。

(2)

将S数组从小到大排序，然后从小到大排序，将两个相邻数之间的最小值作为$|x-y|$的值，这两个数分别作为x和y即可。

2.18

我们先将$d_i=x_i^2+y_i^2$，然后对d进行选择，将其较小的前$\lfloor\sqrt{n}\rfloor$个数选择出来。

复杂度为$T(n)=O(n)+O(\sqrt{n}\log\sqrt{n})=O(n)$

伪代码如下：

```
for i in 1 .. n: D[i] = x[i]*x[i] + y[i]*y[i];
i = floor(sqrt(n))
x = quick_Seleck(D, i); // 选取第i小的数
Swap(D, x, i); // 将比x小的数交换到i前面
sort(D + 1, D + i + 1); // 将前i个数排序，记录下他们的标号
Print(D, i);
```

2.23

我们将序列的第$\frac{n}{4}$小的数a找到，和第$\frac{3n}{4}$小的数b找到，然后把序列划分成$x\le a, a<x<b, x\ge b$三种。

- 如果$a=b$，则不存在近似中值。
- 否则在$a<x<b$中的任意数都是近似中值。

2.27

考虑对于x轴和y轴的选取是独立的，也就是说x和y是不互相影响的，我们先只对一个x轴计算.

$ans = \min \sum_{i=1}^{n} |x - x_i|$，不难发现，我们只需要选取$x_i$的中位数即可。对$y$轴同理。

所以我们只要$O(n)$的计算x和y的中位数即可。这个(x, y)就是仓库的位置。

算法时间复杂度：$O(n)$。

补充题

1.

> 设在一个环上排列这n个互不相同的数。设计一个比较次数尽可能少的算法，寻找环上的一个极小值（即比左右两个相邻的数都小的数）。

考虑在环上面三分。取三分点为p_1、p_2和p_3。考虑a_{p_1}、a_{p_2}和a_{p_3}互不相等，所以我们可以找到这三个数中的极小值；

存在如下定理：

> 如果存在一个$i < j < k, a_i > a_j, a_k > a_j$，那么在$(i, k)$这段区间中一定存在一个极值点。

所以我们将剩下两个值中间的劣弧从环中删掉，然后对剩下的环继续做这个分治操作即可。

比较次数为：$O(3 \times \log_{\frac{3}{2}} n) = O(\log n)$

2.

> 对于芯片问题的分治算法中，当对芯片成对分组时，如何避免对轮空芯片的单独测试，仍能保证找到好芯片？结合子问题的定义和循环不变量，证明改进算法的正确性。

我们考虑在检测完成组的芯片后剩下的芯片如果是偶数个，就保留轮空芯片；否则直接移除轮空芯片

正确性证明：

我们不妨设一开始有2n+1个芯片，移除一个芯片后剩下的$2n$个芯片做分组测试，这其中一共有a个好芯片,b个坏芯片。

然后测试完之后还剩下a'个好芯片，b'个坏芯片， 则：

- $a' == b'$，很明显要求$a == b$且移除的芯片为好芯片，此时剩余的芯片总数为$a' + b' = 2a'$,是偶数，所以我们保留轮空芯片，仍然满足好芯片个数大于坏芯片个数。即$a' + 1 > b'$.
- $a' > b'$
  - 如果剩下芯片个数是奇数个，我们直接移除，仍然能够保证满足好芯片个数大于坏芯片个数
  - 如果剩下芯片个数是偶数个，则有$a' > b' + 1$，因为a', b'都是整数所以一定成立。所以我们即使保存了一个坏芯片，也能证明剩余的好芯片个数多于坏芯片个数。

所以综上所述，按照这样的方法来对轮空芯片取舍能够保证答案的正确性。